
GERAYO · PRODUCT ENGINEERING WALKTHR

# Tap&Go: h contactless fare tap-in / tap-out works in Gerayo

A workflow demonstration of the Tap&Go card feature — from enrollment to boarding to top-up — as implemented in the current codebase.

| FEATURE | SURFACE | STATUS |
| --- | --- | --- |
| **Tap&Go transit card** | **Gerayo web app** | **Shipped —** `main` |

CONTEXT

# Why Tap&Go exists

Gerayo already sells seat tickets for scheduled intercity routes. Tap&Go adds a second, lighter-weight fare model for city transit: no seat selection, no advance purchase — riders load a reusable card and tap at the reader.

Problem

### Cash friction on short hops

City routes involve frequent, low-value trips where per-ride ticket purchase is too slow for both rider and driver.

Approach

### Prepaid balance card

Riders enroll a card ID once, top it up via mobile money / agent / bank, then tap to board — fare is deducted instantly.

Scope

### Simulated NFC tap

The current build models the reader interaction end-to-end in the UI, backed by `localStorage` — ready to be wired to real hardware / a backend.

ARCHITECTURE

# Where Tap&Go lives in the codebase

One hook owns all card state; every screen is a thin view over it.

hook

**useTapGoCard()**

`src/hooks/useTapGoCard.js` — cards, active card, balance, transactions, enroll / topUp / tapBoard / tapExit

↔

mounted in

**App.jsx**

Instantiates `tapGo` once and passes it into `TicketsTapGoView`

view

**TapGoCardView**

Balance card + route picker

modal

**TapGoEnroll**

First-time card registration

modal

**TapGoTapModal**

Boarding tap animation + charge

modal

**TapGoTopUpModal**

Add balance by method

moda

**Tap**

Mult

`localStorage:gerayo_tapgo_cards` `localStorage:gerayo_tapgo_active_card` `localStorage:gerayo_tapgo_archived_cards`

WORKFLOW

# The rider journey, end to end

Six steps take a rider from no card to a completed, deducted trip.

01

**Enroll a card**

Rider enters a card ID (or uses the demo card) to register it.

`TapGoEnroll.jsx`

02

**View balance**

Card renders with live RWF balance and identifier.

`TapGoCardView.jsx`

03

**Top up**

Choose a preset amount and a payment method.

`TapGoTopUpModal.jsx`

04

**Select a route**

Rider picks a city route to board.

`TapGoCardView.jsx`

05

**Tap to board**

Reader detects the card; fare is deducted.

`TapGoTapModal.jsx`

06

**Tap to exit**

Trip closes out; a new transaction is logged.

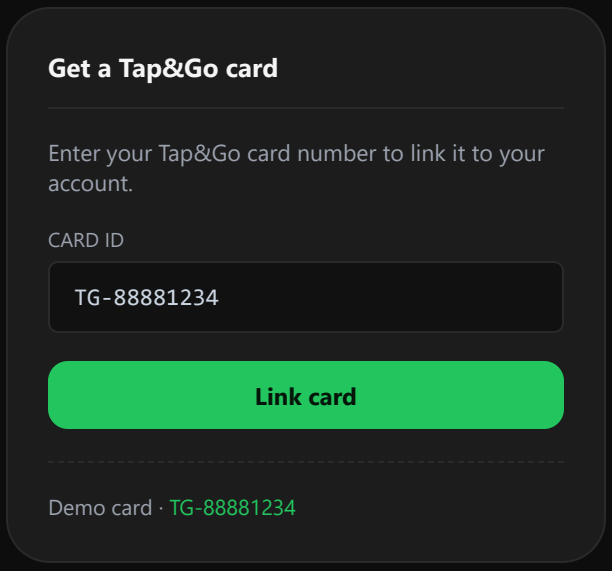
`tapExit()`

**State owner**

Every step above reads and writes the same `tapGo` object returned by `useTapGoCard()` — there is no separate state per screen, so balance and history stay consistent across modals.

01 ENROLLMENT

# Registering a card for the first time




a **ID normalized & validated**
Input is trimmed and upper-cased before matching; empty IDs are rejected client-side.

b **Re-enroll restores archive**
If the ID matches a previously removed card, `enroll()` restores it from `archivedCards` instead of creating a duplicate.

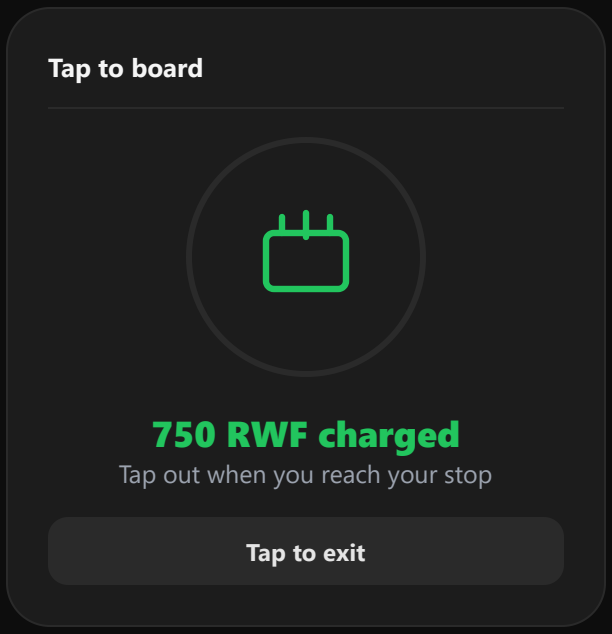
c **New cards start funded**
Unknown IDs create a fresh card with a starting balance of `1,500 RWF` and an empty transaction log.

d **Becomes the active card**
Whichever ID succeeds is immediately set as `activeCardId`, so the balance screen updates without extra taps.

02 BOARDING

# Tap to board: the reader simulation

Selecting a route opens `TapGoTapModal`, which walks through three timed states before confirming the charge.




1 **waiting — 0ms**
Modal opens in a neutral grey state, holding until the simulated reader detects the card.

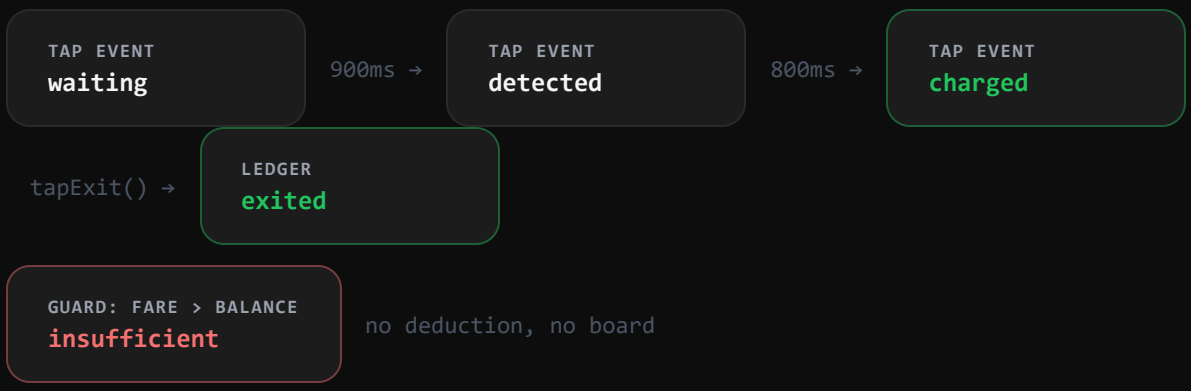
2 **detected — 900ms**
Ring pulses green; label shimmers to `Tap detected` while the transaction resolves.

3 **charged — 1700ms**
`onConfirmBoard()` fires: `tapBoard(route)` deducts the fare and logs a `boarded` transaction.

⊗ **Insufficient balance guard**
If `route.fare > balance`, the flow skips straight to a red error state and no deduction happens.

BEHIND THE SCENES

# Fare state machine & the transaction ledger




Every balance change writes a matching entry to `card.transactions`, giving Tap&Go a full audit trail per card:

| ACTION | FUNCTION | BALANCE EFFECT | TRANSACTION TYPE LOGGED |
|---|---|---|---|
| Add funds | topUp(amount, method) | + amount | `topup` |
| Board a route | tapBoard(route) | − route.fare | `boarded` |
| Exit at destination | tapExit(route) | no change | `exited` |
| Direct spend (e.g. parcel) | spend(amount, meta) | − amount | `boarded` |

03 MANAGING MONEY & CARDS

# Top-up methods and multi-card support

`TapGoTopUpModal.jsx`

### Four funding methods

Preset amounts (500 / 1,000 / 2,000 / 5,000 RWF) plus a choice of channel:

🧍 Agent · 📱 MoMo · 📶 Airtel · 💳 Bank

Confirming calls `topUp(amount, method)`, which credits the active card and appends a `topup` transaction.

`TapGoManageCardsModal.jsx`

### One rider, several cards

Cards are stored as a list, not a singleton, so the modal supports:

- Enrolling an additional card without losing the first
- Switching the active card (`switchCard`)
- Renaming a card's label (`renameCard`)
- Removing a card — it moves to `archivedCards`, not deleted, so re-enrolling restores its balance and history

### TapGoHistory.jsx

Below the balance card, every transaction for the active card renders newest-first: top-ups show in green with a `+` prefix, boardings show the origin → destination pair, all timestamped with `toLocaleTimeString()`.

SUMMARY

# What's shipped, and what's next

**1**
Hook owns all state — `useTapGoCard()`

**6**
Screens: enroll, card view, tap, top-up, manage, history

**4**
Transaction types tracked per card

**3**
localStorage keys persist cards across sessions

## NATURAL NEXT STEPS

**BACKEND**
Replace `localStorage` with a synced account service so balance survives device changes.

**HARDWARE**
Connect `TapGoTapModal`'s simulated timing to a real NFC/RFID reader event.

**PAYMENTS**
Wire the four top-up methods to live MoMo / Airtel / bank payment rails.